UNIVERSIDAD CATÓLICA ANDRÉS BELLO
FACULTAD DE INGENIERÍA
Escuela de Ingeniería Informática
Unidad Curricular: Sistemas Operativos
Term: 202715

Taller N° 1.

Procesos e Hilos. Creación y Sincronización

Modalidad: Trabajo en equipo (Máximo 3 integrantes)
Duración: 45 Minutos (1 Hora Académica)

1. Objetivos de Aprendizaje

- Aplicar comandos de consola en sistemas operativos para la inspección, control y finalización de procesos activos.
- Desarrollar scripts de automatización por lotes para la gestión secuencial de procesos en el entorno operativo.
- Implementar hilos de ejecución mediante la librería estándar POSIX Threads (pthread) en lenguaje C.
- Coordinar el orden estricto de ejecución inter-hilos mediante semáforos POSIX (sem_t).
- Garantizar la exclusión mutua durante el acceso y modificación de recursos compartidos en memoria mediante semáforos Mutex (pthread_mutex_t).
- Utilizar flujos de trabajo colaborativos con Git y GitHub (ramas, Pull Requests y merges) para la construcción modular de software en equipo.

2. Herramientas Requeridas

- **Sistema Operativo:** Microsoft Windows (consola PowerShell / CMD).
- **Entorno de Desarrollo:** Visual Studio Code (VS Code) con terminal integrada.
- **Compilador C:** MinGW-w64 / GCC (incluido en el paquete portable w64devkit).
- **Control de versiones:** Git local y plataforma GitHub.
- **Asistente de IA:** Google Gemini (en modo de tutoría socrática).

3. Preparativo Inicial (3 minutos)

---

1. Abran la terminal integrada en VS Code (Ctrl + ~) y verifiquen la presencia de GCC:
   `gcc --version`
2. **Si GCC no es reconocido:** Descarguen el paquete portable w64devkit desde el enlace de Drive facilitado por el profesor, extraigan su contenido e integren la ruta temporalmente en la terminal:
   `$env:Path += ";C:\w64devkit\bin"`

4. Desarrollo

PARTE 1: Inspección de Procesos y Scripting (10 Minutos)

Actividad 1.1: Descubrimiento Socrático con IA (4 min)

Abran una sesión en **Gemini** y utilicen el siguiente prompt inicial:
"Actúa como un profesor y tutor socrático en Sistemas Operativos. No me des la respuesta directa ni me des el código listo. Guíame mediante preguntas para que yo descubra cómo listar procesos en tiempo real en la consola CMD de Windows, cómo lanzar una aplicación como el Bloc de Notas (notepad.exe) desde la terminal y cómo finalizar dicho proceso usando su nombre o su PID. Hazme una pregunta a la vez."
Demostración Práctica: Muestren al profesor la ejecución en CMD o PowerShell lanzando un Bloc de notas, identificando su PID y finalizándolo por consola.

Actividad 1.2: Script de Automatización por Lotes (6 min)

Creen un archivo llamado lanzar_procesos.bat (o .ps1) que realice lo siguiente:
1. Lance 3 instancias del Bloc de Notas (notepad.exe).
2. Muestre en pantalla la lista de procesos filtrados por notepad.exe.
3. Espere 5 segundos automáticamente.
4. Finalice las 3 instancias de notepad.exe de forma forzada.

PARTE 2: Sincronización y Trabajo Colaborativo en C (32 Minutos)

Actividad 2.1: Práctica Guiada de Git y Ejecución Inicial (10 min)

1. **Clonación y Configuración:**
   - **Integrante 1:** Descarga de Drive el archivo de ejemplo sync_posix.c, crea el repositorio Taller-SO en GitHub y sube el archivo a la rama main.
   - **Integrantes 2 y 3:** Clonan el repositorio en sus equipos.
2. **Personalización por Ramas:**
   - Cada integrante crea una rama propia (feature/hilo-a, feature/hilo-b, feature/hilo-c).
   - Modifican el código para que al inicio de los mensajes de cada hilo aparezca el nombre del integrante encargado:
     `printf("[Hilo A - Dev: Nombre Apellido] Paso 1: Iniciando...\n");`
3. **Integración:** Realizan git push, abren el *Pull Request* hacia main y el Integrante 1 consolida los cambios.
4. **Prueba de Compilación:** Todos ejecutan en la terminal de VS Code:

```
gcc sync_posix.c -o sync_posix.exe -lmingw32 -mconsole -pthread
.\sync_posix.exe
```

Actividad 2.2: La Prueba de Fuego – Intercambio y Suma Sincronizada (22 min)

Deben coordinar 3 hilos que comparten una única variable global llamada variable_compartida bajo dos principios:

1. **Sincronización de Secuencia (sem_t):** Orden de ejecución estricto A → C → B → C.
2. **Exclusión Mutua (pthread_mutex_t):** Proteger el acceso a variable_compartida para evitar condiciones de carrera en cada lectura o escritura.

Reglas de Desarrollo:

- **Hilo A (Integrante 1):** Asigna un valor numérico a variable_compartida (protegido por Mutex) y notifica a Hilo C.
- **Hilo C (Integrante 3 - Parte 1):** Espera a Hilo A, lee y guarda localmente Cifra 1, limpia variable_compartida = 0 (protegido por Mutex) y notifica a Hilo B.
- **Hilo B (Integrante 2):** Espera a Hilo C, asigna un nuevo valor numérico a variable_compartida (protegido por Mutex) y notifica a Hilo C.
- **Hilo C (Integrante 3 - Parte 2):** Espera a Hilo B, lee Cifra 2 (protegido por Mutex), calcula la suma e imprime en pantalla el desglose final.

Compilación final:

```
gcc taller1_so.c -o taller1_so.exe -lmingw32 -mconsole -pthread
.\taller1_so.exe
```

5. Rúbrica de Evaluación

Criterio de Evaluación	Descripción e Indicadores	Puntaje Máximo
1. Inspección y Scripting (Parte 1)	• Demostración correcta con Gemini en modo socrático de los comandos tasklist / taskkill y apertura de procesos (2 pts). • Script ejecutable (.bat o .ps1) que automatiza el lanzamiento, listado y cierre forzado de instancias (2 pts).	**4 Puntos**
2. Control de Versiones con Git/GitHub	• Creación correcta del repositorio base en GitHub y estructura limpia en main (1 pt). • Uso efectivo de ramas por cada integrante, envío de Pull Requests e integración exitosa sin conflictos (2 pts). • Mensajes de commit descriptivos y participación equilibrada de los 3 integrantes (1 pt).	**4 Puntos**
3. Sincronización de Hilos POSIX	• Uso correcto de semáforos POSIX (sem_t, sem_init, sem_wait, sem_post) (3 pts).	**5 Puntos**

Criterio de Evaluación	Descripción e Indicadores	Puntaje Máximo
	• Cumplimiento estricto del orden secuencial de ejecución (A → C → B → C) (2 pts).	
4. Exclusión Mutua con Mutex	• Implementación adecuada del Mutex (pthread_mutex_t, pthread_mutex_lock, pthread_mutex_unlock) para proteger las lecturas y escrituras en la variable compartida (3 pts). • Ausencia de condiciones de carrera (*Race Conditions*) o interbloqueos (*Deadlocks*) (2 pts).	**5 Puntos**
5. Compilación y Resultados	• Compilación limpia en la terminal de VS Code con GCC utilizando las banderas apropiadas (-lmingw32 -mconsole -pthread) (1 pt). • Salida correcta por consola mostrando la Cifra A, Cifra B y la Suma Total calculada por el Hilo C (1 pt).	**2 Puntos**
	TOTAL:	**20 Puntos**